BUSINESS INTELLIGENCE AND DATA ANALYTICS

Institute of Analytics
Recognized Programme

University of Petra

307401 Big Data

Introduction to AWS Data Engineering

2024-2

Ingesting Data by Batch or Stream

Ingesting by Batch or Stream

Objective: Understand how to design, build, and scale data ingestion pipelines using AWS services.

You will learn to:

- Identify the tasks involved in building an ingestion layer.
- Compare batch and stream ingestion strategies.
- Understand AWS Glue for batch ingestion.
- Learn about Amazon Kinesis for stream ingestion.
- Explore ingestion for IoT data using AWS IoT Core and AWS IoT Analytics.
- Address performance and scaling considerations in ingestion pipelines.

Comparing Batch and Stream Ingestion

Batch Ingestion:

- Collects and processes data in fixed-size datasets.
- Triggered on demand, by schedule, or events.
- Ideal for periodic reporting and aggregated analytics.

Streaming Ingestion:

- Continuously collects data in near real-time.
- Processes data immediately upon arrival.
- Suitable for real-time dashboards, recommendations, and alerts.

Key Decision Factors:

- **Volume:** Total amount of data to ingest.
- **Velocity:** Frequency at which data arrives.
- **Use Case Examples:**
  - Batch: Retail sales aggregated daily.
  - Stream: Website clickstream analysis.

Batch Ingestion Pipeline – Tasks and Considerations

Common Steps:

- **Extract:** Read from source systems.
- **Transform/Load:** Clean and modify data.
- **Load/Transform:** Load into target system and transform in place.

Design Requirements:

- **Ease of Use:** Low-code or no-code tools; serverless options.
- **Scalability:** Ability to handle large, diverse data volumes.
- **Monitoring:** Workflow orchestration, logging, alerting.
- **Cost Optimization:** Pay-as-you-go models and auto-scaling.

Purpose-Built AWS Tools for Batch Ingestion

Tool	Best For	Example Use Case
Amazon AppFlow	SaaS data integration	Ingesting support tickets from Zendesk
AWS DMS	Relational database migration	Oracle to Amazon Redshift data migration
AWS DataSync	File system data transfer	Genome files to Amazon S3
AWS Data Exchange	Ingesting third-party datasets	Clinical health data from data providers

AWS Glue – Simplifying Batch Ingestion

What it Offers:

- **Schema Detection:** Crawlers infer schema from raw data.
- **Central Catalog:** AWS Glue Data Catalog stores metadata.
- **Job Authoring:** Visual ETL development with Glue Studio.
- **Serverless ETL:** Spark-based jobs without managing servers.
- **Orchestration:** Workflows with triggers, dependencies, retries.
- **Monitoring:** CloudWatch insights, Spark UI, job logs.

Scaling Batch Processing with AWS Glue

Performance Goals:

- Complete within time limits, stay within budget, and reduce failures.

Scaling Options:

- **Horizontal Scaling:** Add more Glue workers. Best for large datasets.
- **Vertical Scaling:** Use more powerful workers. Ideal for memory-heavy tasks.

Optimization Tip:

- Use fewer large files (splittable) over many small files to reduce overhead.

Streaming Ingestion with Kinesis

Tasks in a Streaming Pipeline:

- **Extract:** Producers push events to stream.
- **Transform/Load:** Consumers process and store data.

Design Features:

- **High Throughput:** Handle large-scale, fast-moving data.
- **Loose Coupling:** Producers and consumers work independently.
- **Parallelism:** Multiple consumers can process the same stream.
- **Fault Tolerance:** Replay events and checkpoint processing progress.

Purpose-Built Kinesis Services

Service	Role in Pipeline
Kinesis Data Streams	Store and transport data in real-time shards
Kinesis Firehose	Apply transformations and load into storage (e.g., S3)
Kinesis Data Analytics	Run SQL/Java/Python/Scala analytics in real time

Integration: Ingests from IoT devices, web sources, apps, or other AWS services.

Scaling Stream Processing

Key Configurations:

- **Retention:** Define how long data stays on stream.
- **Write Capacity:** Set throughput for producers.
- **Read Capacity:** Optimize for consumer performance.

Monitoring Tools:

- **CloudWatch:** Track latency, failures, throttling.
- **CloudTrail:** Audit configuration changes and data access.

IoT Data Ingestion – AWS Services

AWS IoT Core:

- Connect, authenticate, and route data from IoT devices.
- Supports MQTT protocol and publish/subscribe model.
- Routes data via rules engine to S3, Kinesis, Lambda, etc.

AWS IoT Analytics:

- Ingests unstructured IoT data.
- Transforms and stores time-series data.
- Enables visualization and analysis.
- **Architecture Tip:** Use IoT Core for ingestion, and IoT Analytics for transformation and storage.

Storing and Organizing Data

Storing and Organizing Data

Objective: Learn how to select, use, and secure cloud-based storage solutions in a modern data architecture.

By the end of this module, you will be able to:

- Define key storage types used in cloud data architectures.
- Differentiate between data lakes, data warehouses, and purpose-built databases.
- Match storage types with data processing needs.
- Implement security best practices for cloud-based storage.

Types of Storage in Cloud Data Architectures

Block Storage:

- Provides low-latency, high-performance storage blocks.
- Suitable for databases and boot volumes.
- Example: **Amazon Elastic Block Store (EBS).**

File Storage:

- Organizes data in a hierarchical file system.
- Ideal for shared access to media, content, and backups.
- Example: **Amazon Elastic File System (EFS).**

Object Storage:

- Stores data as individual objects with metadata and unique identifiers.
- Supports massive scalability and cost efficiency.
- Example: **Amazon Simple Storage Service (S3).**

Understanding the Storage Layer

- The **storage layer** is the foundation of any data pipeline.
- It integrates natively with AWS services and enables scalable storage for various data types.
- Choosing the right storage type ensures performance, cost-effectiveness, and reliability.

Data Lake Storage

What is a Data Lake?

- A centralized repository for storing structured, semi-structured, and unstructured data.
- Allows you to store data in its raw form without requiring a predefined schema.
- Built using **Amazon S3** as the foundational storage layer.

Core Features:

- Unlimited scale and low-cost storage.
- In-place querying using services like Athena or Redshift Spectrum.
- Integration with analytics and ML tools.
- Secure data access using **Lake Formation**.

Amazon S3 – Data Lake Backbone

- Highly durable, cost-effective, and secure.
- Offers **storage classes** (e.g., Standard, Glacier) for cost optimization based on access frequency.
- Supports **multipart uploads** and **strong consistency**.
- Can serve as both active storage and archival.

Amazon Lake Formation:

- Simplifies creation and management of secure data lakes.
- Adds support for ACID transactions using governed tables.
- Extends IAM with fine-grained access control.

Data Warehouse Storage

What is a Data Warehouse?

- A centralized system for storing structured and semi-structured data optimized for analytics.
- Separates storage from transactional systems to focus on querying and reporting.

Amazon Redshift:

- Fully managed, cloud-native data warehouse.
- Uses **columnar storage** for high-speed queries.
- Supports **Redshift Spectrum** to query S3 data directly.

Data Lakes vs. Data Warehouses

Feature	Data Lake (S3)	Data Warehouse (Redshift)
Data Type	Structured, semi-structured, unstructured	Structured, semi-structured
Schema	Schema-on-read	Schema-on-write
Storage Cost	Lower	Higher
Use Case	ML, raw data, flexible exploration	BI, dashboards, structured analytics
Query Engines	Athena, EMR, Redshift Spectrum	Redshift

Purpose-Built Databases

Choosing a Purpose-Built Database Depends on:

- **Application Workload:** OLTP, caching, analytics, etc.
- **Data Shape:** Tabular, document, graph, key-value, etc.
- **Performance Requirements:** Query latency, data volume.
- **Operations Burden:** Backup, scaling, recovery, failover.

Examples of AWS Purpose-Built Databases:

- **RDS:** Relational data (MySQL, PostgreSQL, etc.)
- **DynamoDB:** NoSQL key-value and document.
- **Aurora:** High-performance relational DB.
- **Neptune:** Graph databases.

Storage in Support of ETL vs. ELT Pipelines

ETL (Extract, Transform, Load):

- Data is transformed before being stored.
- Typically uses intermediate storage during processing.

ELT (Extract, Load, Transform):

- Data is loaded into the storage first.
- Transformation is done after storage, enabling full flexibility and reusability.

Storage Types in Pipelines:

- **ETL:** Uses temp storage and targets like S3 or Redshift.
- **ELT:** Loads raw data directly into S3 or Redshift and transforms it using SQL or Glue.

Securing Cloud Storage

Amazon S3 Security Features:

- Resource-based and user IAM policies.
- Encryption options: SSE-S3, SSE-KMS, client-side encryption.
- Fine-grained access using S3 Access Points and tags.
- Governed access via Lake Formation.

Amazon Redshift Security:

- Handles **service-level** and **database-level** security separately.
- Integrates with **CloudWatch, CloudTrail**, and **Security Hub** for auditing.
- Supports compliance and governance requirements for regulated workloads.

Processing Big Data

Processing Big Data

Objective: Learn to select, configure, and operate big data processing frameworks using AWS tools.

By the end of this module, you will be able to:

- Compare batch vs. streaming big data processing.
- Understand Apache Hadoop and Apache Spark.
- Deploy and manage Amazon EMR clusters.
- Use Apache Hudi for near real-time data updates.

Big Data Processing Concepts

Types of Processing:

Batch Processing:

- Handles large volumes of historical (cold) data.
- Processes data in scheduled intervals or batches.
- Suitable for deep, complex analysis.
- Tools: Amazon EMR, Hadoop, Spark (batch mode).

Streaming Processing:

- Ingests and processes real-time (hot) data as it arrives.
- Handles continuous, unpredictable data flows.
- Supports low-latency analytics and event-driven systems.
- Tools: Apache Spark Streaming, Amazon Kinesis.

Apache Hadoop Overview

What is Hadoop?

- An open-source distributed framework for storage and processing.
- Executes jobs across clusters of servers.

Core Components:

- **HDFS (Hadoop Distributed File System):** Stores data in blocks with replication.
- **YARN:** Allocates cluster resources and manages job execution.
- **MapReduce:** Splits jobs into map and reduce tasks for parallel execution.

Hadoop is best for:

- Handling massive datasets with high fault tolerance.
- Traditional batch analytics workloads.

Hadoop Ecosystem Tools

- **Apache Hive:** SQL-like queries on large datasets without writing MapReduce code.
- **Apache Pig:** Data flow scripting for parallel execution.
- **Apache Flink:** Optimized for both stream and batch processing.
- **Presto:** Fast, in-memory query engine ideal for interactive queries.

Apache Spark Overview

What is Spark?

- A distributed data processing framework with in-memory computation.
- Optimized for speed, supports iterative ML and interactive analytics.

Spark Architecture:

- **Driver Program:** Manages the overall job.
- **Executors:** Execute tasks and store data in memory for reuse.
- **SparkContext & Cluster Manager:** Coordinate resources and tasks.

Spark Components:

- Spark Core, Spark SQL, Spark MLlib (ML), Spark Streaming, GraphX.

Amazon EMR – Managed Hadoop and Spark

Amazon EMR (Elastic MapReduce):

- AWS-managed service to run open-source tools like Hadoop, Spark, Hive, and Hudi.
- Supports petabyte-scale data processing.

Use Cases:

- ETL, machine learning, interactive analytics, and log processing.

Node Types:

- **Main Node:** Controls cluster and manages tasks.
- **Core Nodes:** Store data in HDFS and run tasks.
- **Task Nodes:** Execute tasks only.

EMR Cluster Setup and Management

Ways to Launch Clusters:

- Interactive Mode (AWS Console)
- Command Line Interface (CLI)
- APIs (Automation)

Cluster Types:

- **Long-Running Clusters:** Persistent jobs and state.
- **Transient Clusters:** Temporary, shut down after processing.

Scaling EMR:

- **Manual Scaling**
- **EMR Managed Scaling**
- **Custom Auto-Scaling Policies**

Connecting to EMR Clusters

- Connect via SSH to the main node using its public DNS.
- Security groups must be configured for access.
- Clusters must be running to allow any interaction.

Apache Hudi – Managing Dynamic Data

What is Apache Hudi?

- A data management framework optimized for fast ingestion and updates.
- Works well with Spark, Hive, and Presto.

Storage Modes:

- **Copy on Write (CoW):** Writes entire files; ideal for frequent reads.
- **Merge on Read (MoR):** Logs changes separately and merges during read.

Views:

- **Read-Optimized View:** For quick reads.
- **Incremental View:** Shows data changes.
- **Real-Time View:** Combines base files and logs for up-to-date reads.

Real-Time Data Updating with Hudi

- **Hudi DeltaStreamer** automates ingesting and updating datasets.
- Maintains ACID transactions for concurrent data operations.
- Enables schema evolution and data versioning in S3-based lakes.

Analyzing and Visualizing Data

Analyzing and Visualizing Data

Objective: Understand how to choose and apply AWS tools to analyze and visualize data effectively.

By the end of this module, you will be able to:

- Identify factors that influence the choice of analysis and visualization tools.
- Compare AWS services for data analysis and visualization.
- Select the right tool based on business needs, data characteristics, and access requirements.

Factors That Influence Tool Selection

1. Business Needs:

- What decisions must be supported by the analysis?
- What types of insights or KPIs are required?
- Do users need dashboards, reports, or both?

2. Data Characteristics:

- What is the size (volume) and speed (velocity) of the data?
- Is the data structured or unstructured?
- How often is the data updated?

3. Access to Data:

- Where is the data stored, and who needs access?
- What access levels should be granted (read, write, analyze)?
- Follow the principle of least privilege.

Business Needs – Types of Visualizations

Insight Type	Description	Examples
KPIs	Track performance	Revenue, retention rate
Relationships	Show variable interactions	User actions vs. outcomes
Comparisons	Examine differences over time or between groups	Monthly sales by region
Distributions	Display data spread	Error rates, response times
Compositions	Highlight parts of a whole	Sales by category

Data Characteristics – Examples

Data Type	Use Case
Historical data	Drill-down on yearly sales
Real-time (IoT)	Monitor live sensor error rates
Structured	Query customer support logs in RDS
Unstructured	Analyze email sentiment using NLP tools
Mixed sources	Aggregate sales, logs, and social data

Comparing AWS Tools and Services

Tool	Primary Use	Highlights
Amazon Athena	SQL queries on S3 data	Serverless, one-time queries, Apache Iceberg
Amazon QuickSight	Dashboards and BI reporting	Natural language queries with QuickSight Q
Amazon OpenSearch	Real-time log and telemetry analytics	Integrated with OpenSearch Dashboards

Amazon Athena

- Interactive query service using SQL.
- Works directly with data in Amazon S3.
- Serverless—no infrastructure to manage.
- Integrated with BI tools like QuickSight.
- Supports **data modification** via Apache Iceberg (insert, update, delete, version control).

Amazon QuickSight

- Scalable business intelligence (BI) service.
- Connects to various data sources.
- Provides interactive dashboards, KPIs, forecasts.
- Features **QuickSight Q**: Use natural language to ask questions and get instant visual answers.
- **Example:** “Show last year’s weekly sales in California.”

Amazon OpenSearch Service

- Managed open-source search and analytics engine.
- Ideal for log analytics, app monitoring, clickstream analysis.
- Integrated with **OpenSearch Dashboards** (formerly Kibana).

Example Use Case:

- Visualize customer call transcripts using S3 + Comprehend.
- Use pie charts, bar charts, and histograms to track call sentiment and keyword trends.

Use Case – Gaming Analytics

Three Personas:

1. **Analyst:** Uses Athena to query player usage data.
2. **Business User:** Uses QuickSight to visualize revenue and retention KPIs.
3. **DevOps Engineer:** Uses OpenSearch to monitor server performance in real-time.

Key Tools in Use Case:

- **Athena:** For daily aggregated data queries.
- **QuickSight:** For high-level KPI dashboards.
- **OpenSearch:** For live telemetry and log analysis.

Automating the Pipeline

Automating the Pipeline

Objective: Learn how to automate infrastructure and data workflows using DevOps principles and AWS Step Functions.

By the end of this module, you will be able to:

- Identify the advantages of pipeline automation.
- Understand the CI/CD process in data pipelines.
- Describe the components and states of AWS Step Functions.
- Build automated workflows using Step Functions and Athena.

Automating Infrastructure Deployment

Why automate?

- Reduces manual effort, risk of human error, and downtime.
- Ensures repeatable, consistent environment setup.

Infrastructure as Code (IaC):

- Define infrastructure in scripts (JSON, YAML, or Terraform).
- Benefits: **Repeatability** and **Reusability**.
- Automatically recreate environments for testing, deployment, and disaster recovery.

CI/CD and DevOps

CI/CD (Continuous Integration / Continuous Delivery):

- Part of the **DevOps lifecycle**, focused on rapid, reliable software delivery.

CI (Continuous Integration):

- Developers regularly merge code into a shared repo.
- Automated builds and tests validate changes.

CD (Continuous Delivery/Deployment):

- Code changes automatically released to production.
- Reduces risk and speeds up releases.

Benefits in Data Engineering:

- Automates transformation pipelines.
- Reduces delays in deploying analytics or ML updates.

Introduction to AWS Step Functions

What are Step Functions?

- A fully managed **workflow orchestration service**.
- Coordinates components of distributed apps and microservices using **visual workflows**.
- Manages retries, logging, and state transitions automatically.

Key Uses:

- Automate ETL, analytics workflows, and batch processing pipelines.
- Integrate with **Amazon Athena**, **Lambda**, **Glue**, and more.

Step Functions Interface Overview

- **Canvas:** Visual editor where you drag and drop states.
- **States Browser & Actions Panel:** Predefined elements you can use.
- **Inspector:** Displays auto-generated code (JSON/YAML) for each state.

Workflow = State Machine

- Each step = State
- Transitions define the order of operations
- Code is auto-generated but fully customizable

Types of States in Step Functions

State Type	Purpose
Task	Executes work using services like Lambda
Pass	Transfers input to output, no computation
Choice	Branches workflow based on condition
Parallel	Executes multiple branches concurrently
Wait	Pauses workflow for a defined duration
Map	Iterates over items like a loop
Succeed	Marks workflow as successful
Fail	Marks workflow as failed with error

Code Examples – State Types

Pass State Example:

```json
"No-op": {
  "Type": "Pass",
  "Result": { "x": 0.38, "y": 622.22 },
  "ResultPath": "$.coords",
  "Next": "End"
}
```

Choice State Example:

```json
"ChoiceState": {
  "Type": "Choice",
  "Choices": [
    { "Variable": "$.foo", "NumericEquals": 1, "Next": "ChoiceA" },
    { "Variable": "$.foo", "NumericEquals": 2, "Next": "ChoiceB" }
  ],
  "Default": "DefaultState"
}
```

Simplifying ETL with Step Functions

- **Integrates seamlessly** with Amazon Athena and AWS Glue.
- Enables conditional branching to check if Glue tables exist.
- Allows **dynamic workflow creation** based on dataset metadata.
- Supports advanced formats like **Parquet** with **Snappy compression** to optimize storage and query performance.

Lab Summary – Orchestrating ETL with Step Functions

Lab Highlights:

- Use **Step Functions** to coordinate Athena queries.
- Build logic to check for existing tables and create them if needed.
- Store transformed data in compressed Parquet format using **Snappy**.
- Benefit: Reduces storage costs, improves query speed, and supports schema evolution.